Slice internals

Contents

What's actually happening under `[]int{1, 2, 3}`. The header layout, the growth algorithm, the runtime functions, and the performance numbers.

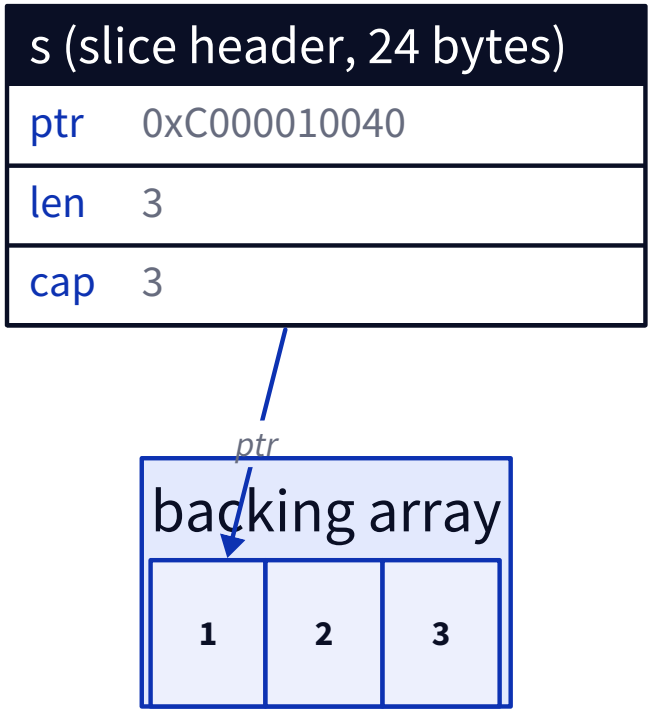
1. The slice header

A slice variable is exactly three machine words. On a 64-bit system, that's 24 bytes total.

```
// runtime/slice.go (simplified)
type slice struct {
    array unsafe.Pointer   // pointer to first element of backing array
    len   int              // number of valid elements
    cap   int              // total allocation size (in elements)
}
```

Every `[]T` variable holds exactly this. The element type `T` is compile-time information; the header doesn't carry it.

```
s := []int{1, 2, 3}
```



Memory layout of `s := []int{1, 2, 3}`

`unsafe.Sizeof(s)` is always 24 (on 64-bit), regardless of element type or slice length.

Inspecting it

```
import "unsafe"

s := []int{1, 2, 3, 4, 5}
hdr := (*struct {
    Data unsafe.Pointer
    Len int
    Cap int
})(unsafe.Pointer(&s))

fmt.Println(hdr.Data, hdr.Len, hdr.Cap)
```

Go 1.20+ gives you the official functions:

```
unsafe.SliceData(s)        // returns *T (the backing array pointer)
len(s)
cap(s)
```

2. How reslicing works

`s[low:high:max]` computes a new header. No data is copied.

```
new.array = s.array + (low * sizeof(T))
new.len   = high - low
new.cap   = max - low      // or (cap(s) - low) without the third index
```

This is why subslicing is O(1) regardless of length. It's just three integer-ish computations.

Bounds checks happen at the call site. The compiler inserts checks that may be elided when it can prove they always pass (loop counters bounded by `len`, etc.).

3. Allocation paths

`make([]T, n)` and `make([]T, n, m)` call the runtime function `runtime.makeslice`.

```
// runtime/slice.go (simplified)
func makeslice(et *_type, len, cap int) unsafe.Pointer {
    mem, overflow := math.MulUintptr(et.size, uintptr(cap))
    if overflow || mem > maxAlloc || len < 0 || len > cap {
        panicmakeslice()
    }
    return mallocgc(mem, et, true)
}
```

`mallocgc` is the central allocator. It picks a size class, finds a span, and returns a pointer. The third argument (`true`) means "needs zeroing."

If escape analysis proves the slice doesn't escape, the compiler skips `makeslice` and stack-allocates the backing array directly. Inspect with:

```
go build -gcflags='-m=2' yourfile.go
```

You'll see lines like `make([]int, 10) does not escape` or `make([]int, n) escapes to heap`.

4. Growth: append

`append(s, x)` has two paths:

Fast path: `len(s) < cap(s)`. Write to `s.array[len]`, return a header with `len+1`. Effectively `s[len] = x; s.len++`.

Slow path: capacity exhausted. The runtime calls `runtime.growslice`, which:

1. Computes a new capacity.

2. Allocates a new backing array via `mallocgc`.
3. Copies old elements with `typedslicecopy` (or `memmove` if elements have no pointers).
4. Returns a new header pointing to the new array.

The old backing array becomes garbage once no other slice references it.

5. Growth algorithm

Go's growth strategy has been tuned several times. As of Go 1.18+, the runtime aims for amortized O(1) append while keeping memory waste under control.

Rough rules (approximate, may change between releases):

Current cap	New cap
0	requested length
< 256	doubled
>= 256	grows by roughly 1.25x, with a floor that adds at least `(oldcap + 3*256) / 4`

The constant 256 is the breakpoint. Below it, doubling keeps the constant factor low. Above it, slower growth keeps memory bloat manageable for very large slices.

After computing the raw new capacity, the runtime rounds up to the nearest size class supported by the allocator (8, 16, 24, 32, 48, 64, 80, 96, …, based on `runtime/sizeclasses.go`). So the actual `cap(s)` after a single append might be slightly larger than the formula suggests.

6. Practical implications

```
s := make([]int, 0)
for i := 0; i < 1000; i++ { s = append(s, i) }
```

Without preallocation, this triggers many reallocations. Each one copies all existing elements. The total cost is O(n), but the constant is much higher than a single allocation.

```
s := make([]int, 0, 1000)   // one allocation, zero copies
for i := 0; i < 1000; i++ { s = append(s, i) }
```

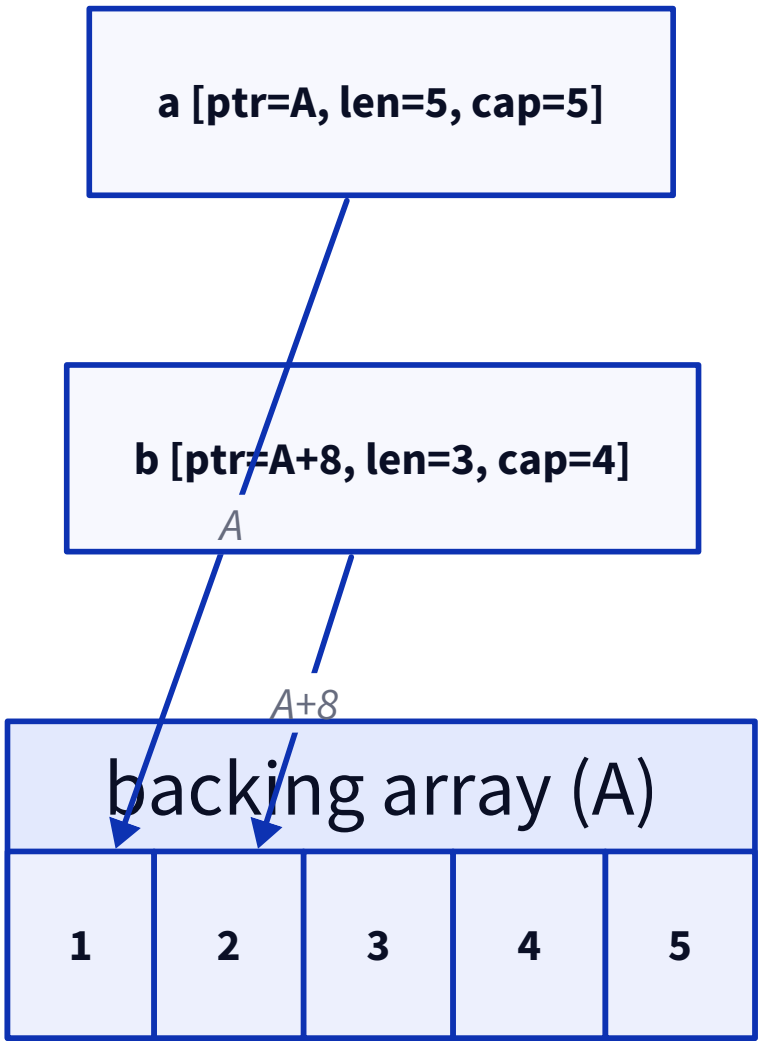
When you know the size, give the hint. Benchmark difference for n=10000:

- Without hint: roughly 13 allocations, around 250 KB total churn
- With hint: 1 allocation, 80 KB

7. The aliasing model

A slice header is independent. Two slice variables can point at the same backing array.

```
a := []int{1, 2, 3, 4, 5}
b := a[1:4]
```



`a` and `b` share the same backing memory

`b[0] = 99` writes to `a.array[1]`. Both slices see the change.

`append(b, ...)` is where it gets tricky:

- If `b`'s capacity (4) has room, append writes into the same backing array. This mutates `a`.
- If `b` exceeds its capacity, append allocates a new array. Mutations to that new array don't touch `a`.

This non-deterministic behavior (in the eyes of someone not thinking about it) is the most-cited slice gotcha.

Three-index slicing as a defense

```
b := a[1:4:4]
```

The third index caps `b.cap` at 4 (the index value minus the low index). Now `b`'s capacity equals its length, so any future append is forced into the slow path and allocates a fresh array.

Use it when handing out a sub-slice that the receiver might append to, and you don't want them stomping on your data.

8. copy

```
n := copy(dst, src)
```

Compiles to a call to `runtime.typedslicecopy` (or `memmove` for pointer-free types). `n` is `min(len(dst), len(src))`.

`memmove` handles overlap correctly. So `copy(s[1:], s[:len(s)-1])` (shift right) and `copy(s[:len(s)-1], s[1:])` (shift left) both work without corruption.

There's no `realloc`. Growth always goes through allocate-then-copy.

9. Slice of pointer types vs slice of values

```
type Big struct { data [1024]byte }

a := make([]Big, 100)          // 100 * 1024 = 102400 bytes contiguous
b := make([]*Big, 100)         // 100 * 8 = 800 bytes of pointers; 100 separate Big
        allocations
```

`a` has better cache locality (one contiguous block) but more zero-fill cost up front. `b` has cheap header allocation but worse iteration cache behavior and more GC pressure (100 separate live objects).

> **Rule of thumb**
>
> Store by value when the type is small or when you iterate hot. Store by pointer when the type is large and elements are rarely accessed together.

10. Garbage collection interactions

The GC scans every pointer-typed element in the backing array. A slice of 1 million pointers means 1 million pointers to scan on every GC cycle.

A slice of pointer-free types (`[]int` , `[]float64` , `[]byte`) is essentially free for the GC to skip over. The element type's pointer map tells the GC whether it can ignore the range entirely.

For high-throughput data buffers, prefer pointer-free element types and pool the slices via `sync.Pool` .

11. The memory leak gotcha

```
func first10(big []int) []int {
    return big[:10]
}
```

The returned slice has `cap = cap(big)` . The backing array is the same array. As long as the caller holds the returned slice, the entire original backing array stays alive, even though only 10 elements are reachable through `len` .

This applies to substrings too. `s[:10]` of a 1 GB string keeps the 1 GB alive.

Fix: copy.

```
out := make([]int, 10)
copy(out, big[:10])
return out
```

12. Zero-length slices

```
var a []int             // nil header: ptr=nil, len=0, cap=0
b := []int{}            // non-nil header: ptr=&runtime.zerobase, len=0, cap=0
c := make([]int, 0)     // same as b
```

All three behave identically in `len` , `range` , and `append` . The runtime has a special `zerobase` symbol that empty allocations point at; this avoids ever returning a nil pointer for `make([]T, 0)` .

The visible difference:

- `a == nil` is true; `b == nil` is false.
- `json.Marshal(a)` produces `null` ; `json.Marshal(b)` produces `[]` .

For most internal code, treat them as equivalent. The runtime does.

13. Bounds checks and BCE

Every `s[i]` is a bounds check unless the compiler can prove it's safe. Failed checks call `runtime.panicIndex` .

Bounds-check elimination (BCE) is the compiler optimization that removes provably-safe checks. The standard idioms it recognizes:

```
// classic loop: BCE applies to s[i]
for i := 0; i < len(s); i++ { _ = s[i] }

// range loop: BCE applies (i and v come from inside)
for i, v := range s { _ = v }

// explicit hint
_ = s[3]                                  // before later s[0], s[1], s[2] in same block
                                          // compiler knows len(s) >= 4
```

If a tight loop indexes a slice with a complex computed index, you can sometimes help the compiler:

```
// before: every s[i+offset] is checked
// after: hoist a sub-slice
sub := s[offset : offset+n]
for i := 0; i < n; i++ { _ = sub[i] }
```

Inspect with `go build -gcflags='-d=ssa/check_bce/debug=1'`.

14. unsafe and string<->[]byte

A string and a `[]byte` have similar runtime representations:

```
type stringHeader struct {
    Data unsafe.Pointer
    Len int
}

type sliceHeader struct {
    Data unsafe.Pointer
    Len    int
    Cap    int
}
```

`[]byte(s)` and `string(b)` both copy. For high-throughput conversions where you can prove the original won't be mutated, Go 1.20+ added safe primitives:

```
b := unsafe.Slice(unsafe.StringData(s), len(s))             // byte view of string (DO
        NOT MUTATE)
s := unsafe.String(&b[0], len(b))                           // string view of bytes (DO
        NOT MUTATE b after)
```

Use these only when profiling shows the copy matters. Mutating either side after the conversion is undefined behavior.

15. Performance characteristics

Operation	Complexity	Notes
`s[i]` read/write	O(1)	one indirection + optional bounds check
`len(s)`, `cap(s)`	O(1)	header read
`s[i:j]`	O(1)	header arithmetic
`append(s, x)` fast path	O(1)	one store, one len increment
`append(s, x)` slow path	O(n) amortized	malloc + memmove
`copy(dst, src)`	O(min(len))	memmove
`len(s)` of nil	O(1)	returns 0
iteration via `range`	O(n)	scalar-friendly; SIMD when element is pointer-free

16. Common runtime functions

Function	When called
`runtime.makeslice`	`make([]T, n)` and `make([]T, n, m)`
`runtime.makeslice64`	length doesn't fit in int
`runtime.growslice`	append slow path
`runtime.slicecopy` (now `typedslicecopy` / `memmove`)	`copy()`
`runtime.panicIndex`	failed bounds check on read/write
`runtime.panicSlice3*`	failed bounds check on `s[low:high:max]`
`runtime.mallocgc`	underlying allocator

Find these in `runtime/slice.go` in the Go source.

17. Worked example: growth trace

```
s := []int{}
for i := 0; i < 10; i++ {
    s = append(s, i)
    fmt.Println(len(s), cap(s))
}
```

Output (Go 1.21):

```
1 1
2 2
3 4
4 4
5 8
6 8
7 8
8 8
9 16
10 16
```

Capacities track the doubling rule below 256. Allocations happen at len = 1, 2, 3, 5, 9 (so 5 reallocations to reach 10 elements). The fix is `make([]int, 0, 10)`: zero reallocations.

18. Best practices

1. Preallocate with capacity when the final size is known or bounded. Even an over-estimate is usually cheaper than reallocation churn.
2. Use three-index slicing at API boundaries when handing out sub-slices that callers might append to.
3. Copy before retaining a small slice carved from a large array. Otherwise the large array stays alive.
4. Prefer slices of values for small types, slices of pointers for large types or polymorphism.
5. Pool slice buffers with `sync.Pool` for short-lived high-throughput allocations.
6. Use `clear(s)` (Go 1.21+) to zero a slice without realloc.
7. Don't take addresses of slice elements that may grow. A subsequent append can invalidate the pointer.
8. Benchmark before reaching for `unsafe`. The string/byte conversion cost is usually negligible.

19. Common gotchas

Gotcha	Cause	Fix
append mutates caller's data	Capacity left over from sub-slicing	Three-index slice
Mystery growth	Default growth rounds up to size class	Inspect `cap`; preallocate
Backing array kept alive	Sub-slice retains full cap	Copy into fresh slice
Pointer to element invalidated	Later append reallocated	Re-fetch by index
nil vs empty serialization	JSON treats differently	Initialize to `[]T{}` if needed
Big slice of pointers, slow GC	Many pointers to scan	Pack into value slice or pool
Out-of-bounds in tight loop	BCE didn't apply	Hoist sub-slice or simplify index
`unsafe.Slice` retained after source freed	Lifetime mismatch	Don't use unsafe unless required

20. Quick reference

Question	Answer
Slice header size on 64-bit	24 bytes (ptr + len + cap)
Backing array allocator	`runtime.mallocgc`
Append fast path	in-place write, no allocation
Append slow path	`runtime.growslice`
Growth below cap=256	doubled
Growth above cap=256	roughly 1.25x
Final cap rounded to	nearest size class
`copy()` implementation	`memmove` (or `typedslicecopy` for pointer types)
Three-index slice purpose	cap the capacity
nil slice vs empty slice	identical for len/range/append; differ in `== nil` and JSON
Bounds check elimination tool	`go build -gcflags='-d=ssa/check_bce/debug=1'`
Escape analysis tool	`go build -gcflags='-m=2'`